Sprint 4: Deployment

TEMA

Alejandro Ciuba
Milan Knezevic
Vaibhav Singh
Daniel Yates

Tech Stack & Deployment

Tech Table

We found this [Medium article] which talked about how a Python backend can communicate with Electron

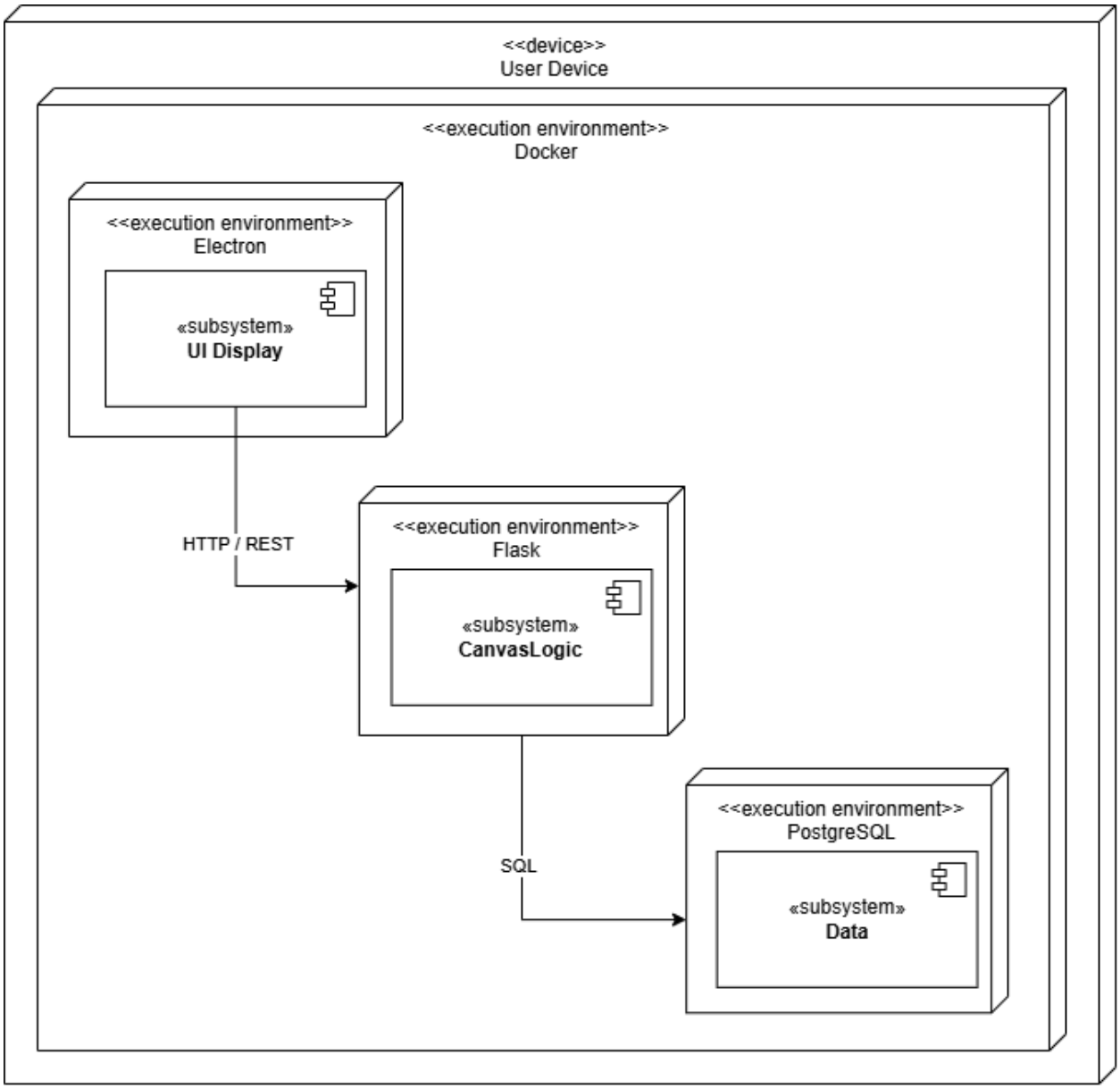
Layer	Technology	Justification
Version Control	Git; GitHub	Git is a robust industry standard for version control, meaning there is plenty of documentation on how to properly maintain a Git repository. GitHub is also a widely available solution for Git repository collaboration. Its robust Actions and features (e.g., tags) will help us maintain TEMA.
Language	Python	Python is a cross-platform programming language with high utility. We can develop most of our software's layers using various Python libraries. Python is also readable and allows for quick prototype development.
Frontend	Electron	Electron is a widely-used frontend for many web applications. Being able to use HTML, CSS and Javascript for our frontend will help us with

		maintainability and quick delivery.
Canvas Logic	Python; Flask	We will use Python to manage the backend logic, and Flask will be our framework for the backend. Flask is a popular web server program in Python.
Data	Postgres via pycopg2	We will save user/canvas data in a Postgres database, which will be accessed through the pycopg2 API for Python. This will let us integrate data operations into our Flask server, which will save time as there will be less overhead since backend logic will be handled in one programming language. Postgres is also used for well-structured relational data, which our canvas data qualifies for.
Testing	pytest; coverage.py; WebDriverIO; GitHub Actions	Pytest is a widely used unit testing library that a team member (Alejandro) has experience in. Additionally, coverage.py can generate reports on the number of executable statements tested, which will help us determine test coverage. WebDriverIO is a web app testing framework recommended in the Electron documentation. GitHub Actions will be used to automatically run relevant tests before merging pull requests to the main branch.
Environment	venv	venv is the built-in Python module for setting up replicable Python environments. This will be used to make sure everyone on the team is working in the

		same version of Python with the same libraries. This also makes our setup more available to those who might want to contribute to our project in the future.
Feature Workflow	Trunk-based	We decided on a trunk-based workflow due to time constraints. This will allow our team to save their work on the GitHub server in their development branches while still being straightforward to merge into the main branch (one pull request with testing). We are not concerned if GitHub commits on development branches are functioning, only that the main branch is.
Deployment	Docker	Docker will be used for deployment as we have several components that need to be started (the Electron frontend, the Python Flask server backend, and the Postgres database). Docker will let us launch all of these in a controlled environment, which also means we will not have to worry about OS-specific installation and launching guidelines for each component.
Development	Docker	Docker will be used as the starting environment our team will work in. This will automatically start the components we need to develop our app in a controlled, replicable environment. Using Docker also means we do not have to worry about component setup across OS's and that all development tools will be

		accounted for and launched once the Docker image is deployed.

Deployment Diagram



Implementation Plan

Sprint Backlog

Task	Description	Assign	Status
[INFRASTRUCTURE] Setup GitHub Repo	Initialize repository, add README, define branching model	Vaibhav/Dan	Done
[INFRASTRUCTURE] Setup repository structure	Setup folders and initial files for fronted, backend, database, and documentation	Dan	Done
[INFRASTRUCTURE] Setup Actions	Setup basic pipeline for GitHub Actions	Vaibhav	WIP
[INFRASTRUCTURE] Setup Docker	Setup development and deployment Dockers	Alejandro	WIP
[INFRASTRUCTURE] Setup Python Environment	Create venv, requirements.txt, and Flask starter application	Dan	Done
[DATABASE] Setup Postgres Database	Configure database container and verify Python backend can connect	Alejandro	TODO
[UI] FR1: **Display Start Page**	**Tag:** User Story 2.Create initial TEMA start page with options to create or open canvas	Milan	TODO
[UI + CANVAS LOGIC] FR1.1 + FR1.2: **Create/Open Canvas**	**Tag:** User Story 2. Add UI and backend logic for creating a new canvas and opening existing canvases		TODO
[UI + CANVAS LOGIC] FR 5: **Drag sticky notes**	**Tag:** User Story 4. Build draggable sticky notes behavior on the canvas		TODO
[UI + CANVAS LOGIC] FR4 + FR6: **Create/Edit Sticky Notes**	**Tag:** User Story 5. Allow user to create sticky notes and edit text, color, and size		TODO
[UI + CANVAS LOGIC +DATA] FR14: **Save Canvas**	**Tag:** User Story 3: Implement functionality that allows the user to save their current canvas so that all sticky notes, positions, edits, and canvas data are stored in the		TODO

	database and can be reopened later		
[UI+ EXPORT LOGIC] FR 12.1: **Export Canvas to PNG**	**Tag:** User Story 6. Implement functionality that allows the user to export the canvas as a PNG image, including the option to export the full canvas or a selected region.		TODO
[TEST] Setup basic testing framework	Configure pytest and create an initial test structure for backend logic	Dan	Done
[TEST] Verify database and backend connection	Test that the Flask backend can successfully connect to PostgreSQL		TODO

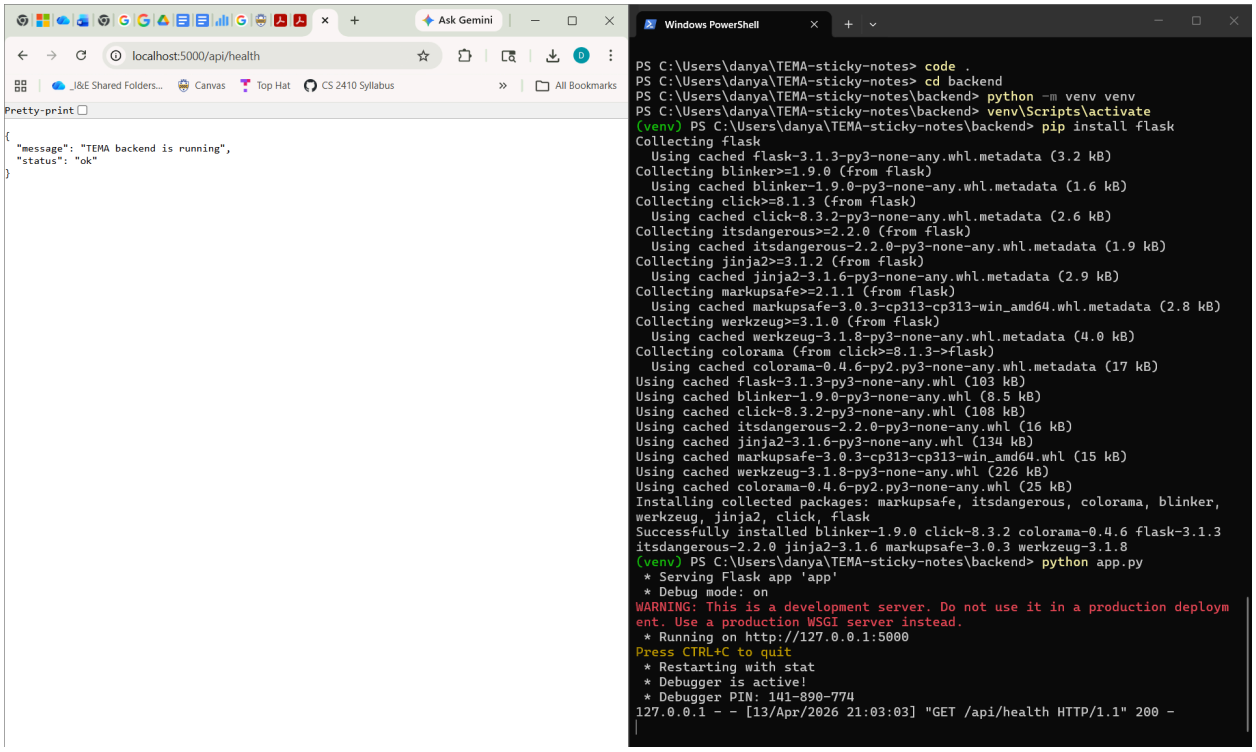
System Infrastructure Setup

Project Setup

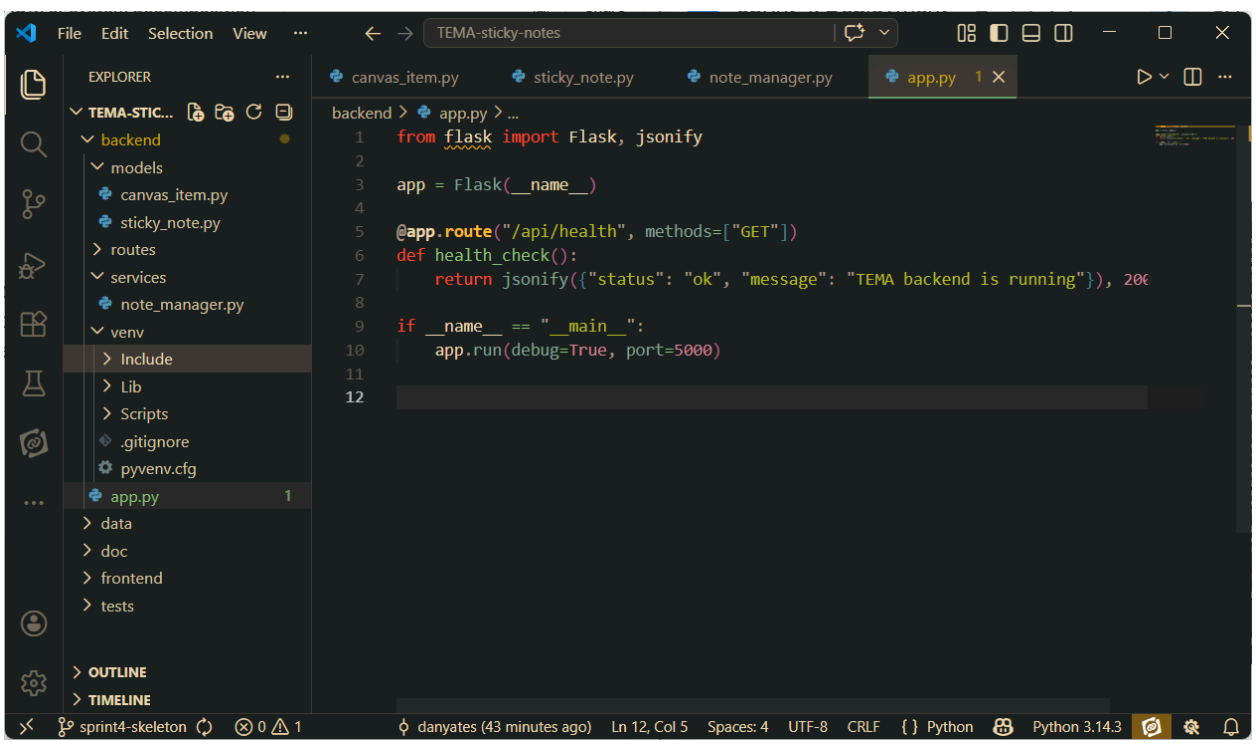
Github Repository: https://github.com/VS511/TEMA-sticky-notes

Our system infrastructure consists of three main folders derived from our subsystem decomposition: frontend (UI subsystem), backend (CanvasLogic subsystem) and data (Data subsystem). We implemented the CanvasLogic subsystem skeleton, located in the backend folder, with subfolders for models, routes, and services. Drawing from our AoM, we implemented canvas_item.py (abstract base class) , sticky_note.py (entity), and note_manager.py (control object) for base functionality to build off of. We also created the Flask application in app.py to test connection to the backend, which we tested and were able to start and receive a response. The database and UI implementation, as well as remaining classes outlined in the AoM, will be left for future sprints.

Confirmation of backend running



File Structure



Design Goals Traceability

Design Goal 1: Usability, Accessibility - Notes can be edited in ≤ 2 clicks
We chose Electron because of its flexibility and implementation in numerous apps similar to TEMA, like Slack, VS Code, and Discord. We can benefit from the convenience of web interfaces in desktop form, helping us to create low-click processes.

Design Goal 2: Fault Tolerance, Response Time - Customizable Autosave
Using Flask allows us to implement an autosave feature that we can intermittently use. Postgres allows us to store structured data that keeps the canvas state stable.

Design Goal 3: UI Responsiveness, Response Time, Utility - No stutter even with > 200 Sticky Notes
By keeping the UI separate from the data logic and data, users can quickly organize and reorganize notes visually without moving lots of data - it lets the program do more work behind the scenes in the Flask application.

Roles

Role	Name	Contribution
Scrum Master (SM)	Daniel Yates	System Infrastructure Setup, Updated Design Goals Traceability, Deployment Diagram
Product Owner (PO)	Alejandro Ciuba	Tech table
Developer	Milan Knezevic	Sprint Backlog
Developer	Vaibhav Singh	Github repo init, Electron frontend development, reviewing